

Practical:3

25IT065

CEUC203

Part A — Practice Programs (Hour 1) Work these in your lab-NN/ folder before the project: do the first one or two in the lab; the last (For Advanced Learners) is for home.

1. Password strength (PasswordChecker.java + Driver.java)

(a) In PasswordChecker, check four rules — length ≥ 8 , contains an uppercase letter, contains a digit, contains a special character — using regex (e.g. “.*[A-Z].*”) or String methods.

(b) Write strength(String pw) that counts how many rules pass and returns “Weak” (0– 1), “Medium” (2–3) or “Strong” (4).

(c) In Driver.main: test several passwords; for each, print which rules passed and the final label. Expected output: “abc” → Weak; “Abcd1234!” → Strong.2.



```
J Driver.java x J PasswordChecker.java
J Driver.java > Driver
1 public class Driver {
2
3     Run | Debug
4     public static void main(String[] args) {
5
6         PasswordChecker pc = new PasswordChecker();
7         String[] passwords = {"abc", "password", "Password", "Password1", "Abcd1234!"};
8         for (String pw : passwords) {
9
10             System.out.println("Password: " + pw);
11             System.out.println("Length >= 8: " + pc.hasMinLength(pw));
12             System.out.println("Contains Uppercase: " + pc.hasUpperCase(pw));
13             System.out.println("Contains Digit: " + pc.hasDigit(pw));
14             System.out.println("Contains Special Character: " + pc.hasSpecialCharacter(pw));
15             System.out.println("Strength: " + pc.strength(pw));
16             System.out.println(x: "-----");
17         }
18     }
```

J Driver.java

J PasswordChecker.java X

J PasswordChecker.java > PasswordChecker

```
1 public class PasswordChecker {
2     public boolean hasMinLength(String pw) {
3         return pw.length() >= 8;
4     }
5
6     public boolean hasUpperCase(String pw) {
7         return pw.matches(regex: ".*[A-Z].*");
8     }
9
10    public boolean hasDigit(String pw) {
11        return pw.matches(regex: ".*[0-9].*");
12    }
13
14    public boolean hasSpecialCharacter(String pw) {
15        return pw.matches(regex: ".*[^a-zA-Z0-9].*");
16    }
17
18    public String strength(String pw) {
19        int count = 0;
20        if (hasMinLength(pw))
21            count++;
22
23        if (hasUpperCase(pw))
24            count++;
25
26        if (hasDigit(pw))
27            count++;
28
29        if (hasSpecialCharacter(pw))
30            count++;
31
```

```
31
32        if (count <= 1)
33            return "Weak";
34
35        else if (count <= 3)
36            return "Medium";
37
38        else
39            return "Strong";
40    }
41 }
```

```

jitpatel2010icloud.com@JitPatels-MacBook-Air prac4 % /usr/
bin/java -XX:+ShowCodeDetailsInExceptionMessages -cp /Users
eStorage/de56377270bfe9bca1eeb7041b5724b4/redhat.java/jdt_w
Password: abc
Length >= 8: false
Contains Uppercase: false
Contains Digit: false
Contains Special Character: false
Strength: Weak
-----
Password: password
Length >= 8: true
Contains Uppercase: false
Contains Digit: false
Contains Special Character: false
Strength: Weak
-----
Password: Password
Length >= 8: true
Contains Uppercase: true
Contains Digit: false
Contains Special Character: false
Strength: Medium
-----
Password: Password1
Length >= 8: true
Contains Uppercase: true
Contains Digit: true
Contains Special Character: false
Strength: Medium
-----
Password: Abcd1234!
Length >= 8: true
Contains Uppercase: true
Contains Digit: true
Contains Special Character: true
Strength: Strong
-----
jitpatel2010icloud.com@JitPatels-MacBook-Air prac4 % █

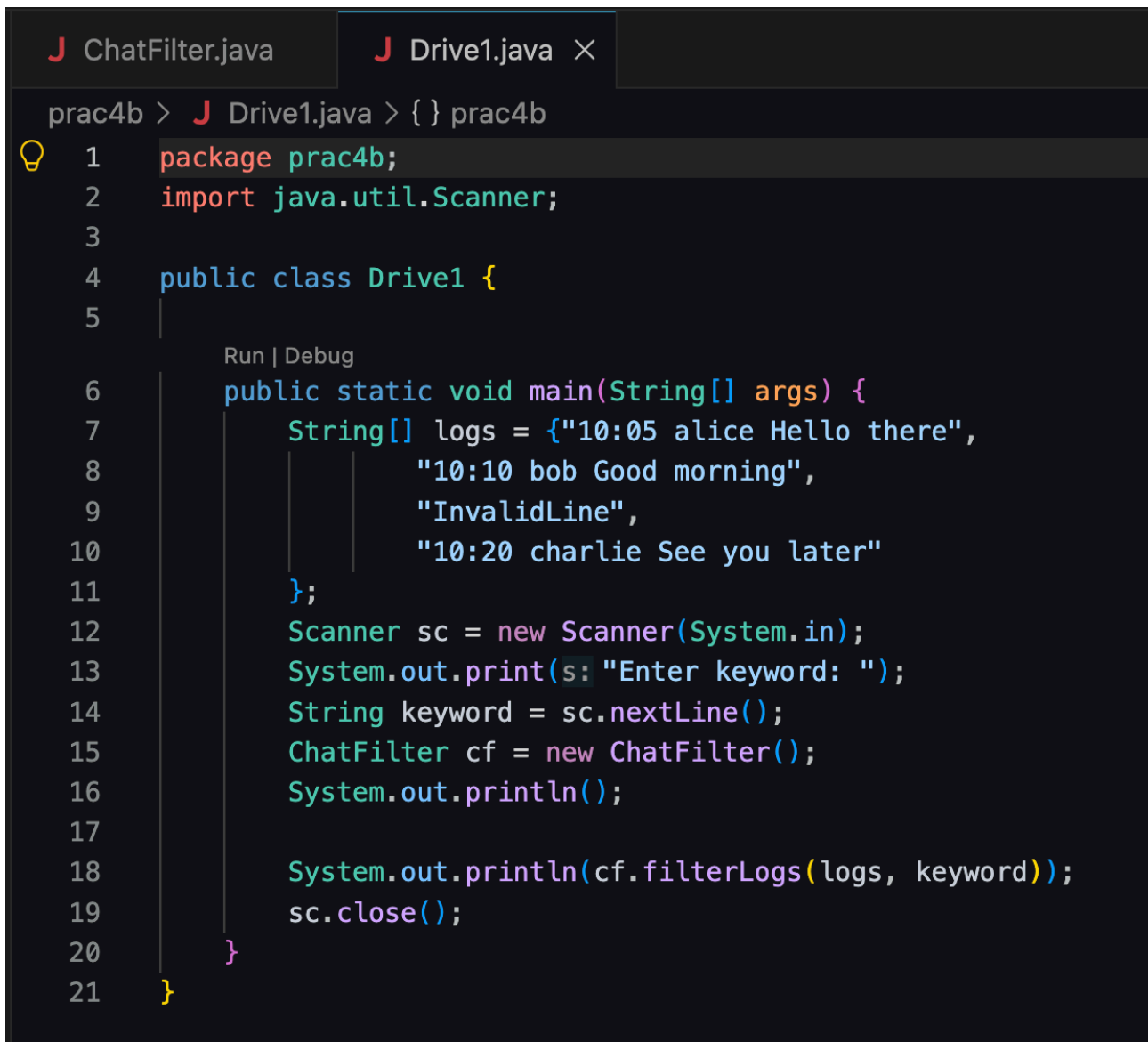
```

2. Chat-log filter (ChatFilter.java + Driver.java)

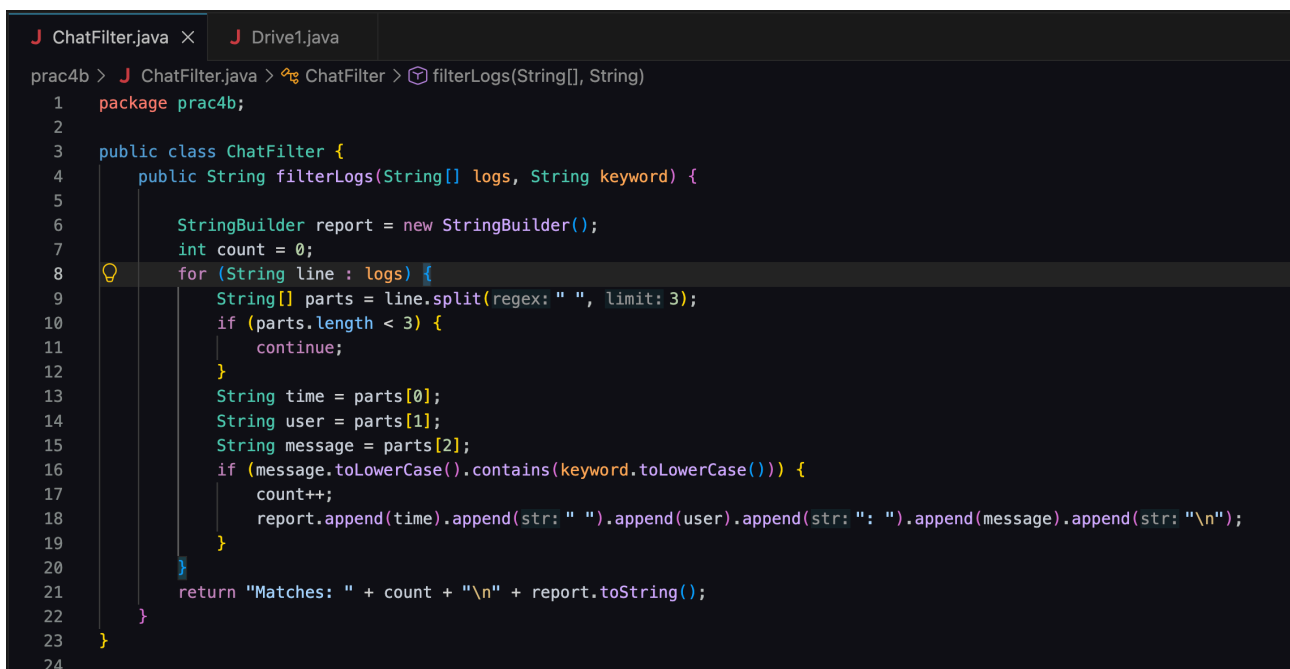
- In Driver.main, define a String[] of log lines like “10:05 alice Hello there” (include one malformed line with fewer than 3 parts).
- Read a keyword from the user.
- For each line, split into time/user/message with split(“ ”, 3); skip lines that do not have 3 parts.
- If the message contains the keyword (case-insensitive), increment a counter and append “time user: message” to a StringBuilder.
- Print “Matches: N” and the built report.Expected output: keyword “hello” over 3 lines (1 malformed) → “Matches: 1” and the matching line.3.

Output:

Output:

A screenshot of an IDE window titled 'J ChatFilter.java' and 'J Drive1.java'. The 'Drive1.java' tab is active, showing a Java program. The code starts with a package declaration 'package prac4b;' and an import 'import java.util.Scanner;'. A public class 'Drive1' is defined with a 'main' method. Inside 'main', an array 'logs' is initialized with four strings: '10:05 alice Hello there', '10:10 bob Good morning', 'InvalidLine', and '10:20 charlie See you later'. A 'Scanner' object 'sc' is created from 'System.in'. The program prompts the user to 'Enter keyword:' and reads a 'keyword' from the input. A 'ChatFilter' object 'cf' is instantiated. Finally, 'cf.filterLogs(logs, keyword)' is called, and the results are printed. The code is as follows:

```
1 package prac4b;
2 import java.util.Scanner;
3
4 public class Drive1 {
5     Run | Debug
6     public static void main(String[] args) {
7         String[] logs = {"10:05 alice Hello there",
8                         "10:10 bob Good morning",
9                         "InvalidLine",
10                        "10:20 charlie See you later"};
11     };
12     Scanner sc = new Scanner(System.in);
13     System.out.print(s: "Enter keyword: ");
14     String keyword = sc.nextLine();
15     ChatFilter cf = new ChatFilter();
16     System.out.println();
17
18     System.out.println(cf.filterLogs(logs, keyword));
19     sc.close();
20 }
21 }
```

A screenshot of an IDE window titled 'J ChatFilter.java' and 'J Drive1.java'. The 'ChatFilter.java' tab is active, showing a Java class. The code defines a 'ChatFilter' class with a 'filterLogs' method. This method takes an array of log strings and a keyword. It iterates through each log line, splits it into time, user, and message parts. If the message contains the keyword (case-insensitive), it increments a counter and appends the log entry to a 'StringBuilder' report. After processing all logs, it returns a string showing the total number of matches and the report. The code is as follows:

```
1 package prac4b;
2
3 public class ChatFilter {
4     public String filterLogs(String[] logs, String keyword) {
5
6         StringBuilder report = new StringBuilder();
7         int count = 0;
8         for (String line : logs) {
9             String[] parts = line.split(regex: " ", limit: 3);
10            if (parts.length < 3) {
11                continue;
12            }
13            String time = parts[0];
14            String user = parts[1];
15            String message = parts[2];
16            if (message.toLowerCase().contains(keyword.toLowerCase())) {
17                count++;
18                report.append(time).append(str: " ").append(user).append(str: ": ").append(message).append(str: "\n");
19            }
20        }
21        return "Matches: " + count + "\n" + report.toString();
22    }
23 }
24 }
```

● <t/Code/User/workspaceStorage/de56377270bfe9bca1eeb7041b5
Enter keyword: Hello

Matches: 1

10:05 alice: Hello there

○ jitpatel2010icloud.com@JitPatels-MacBook-Air prac4 % □

